

Лабораторная №8. Добавление поддержки интерфейса I2C

Цель работы

Изучить I2C, активировать контроллер TWI2 микросхемы Allwinner D1 в Device Tree, сохранить настроенный в лабораторной №7 интерфейс SPI1 и вывести текст на дисплей SSD1306 128×64 из пользовательского пространства.

Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с подготовленной SD-картой;
- компьютер со средой сборки ядра Linux и исходными файлами Device Tree;
- проверенный комплект ядра и DTB из лабораторной №7;
- OLED-дисплей SSD1306 128×64 с интерфейсом I2C;
- четыре соединительных провода;
- USB-UART-преобразователь с логическими уровнями 3,3 В;
- Python 3 и i2c-tools на плате.

Опасность повреждения

Подключайте дисплей только при полностью выключенном питании. Питайте его от 3,3 В и проверьте маркировку контактов конкретного модуля. Не подключайте подтяжки SDA и SCL к 5 В.

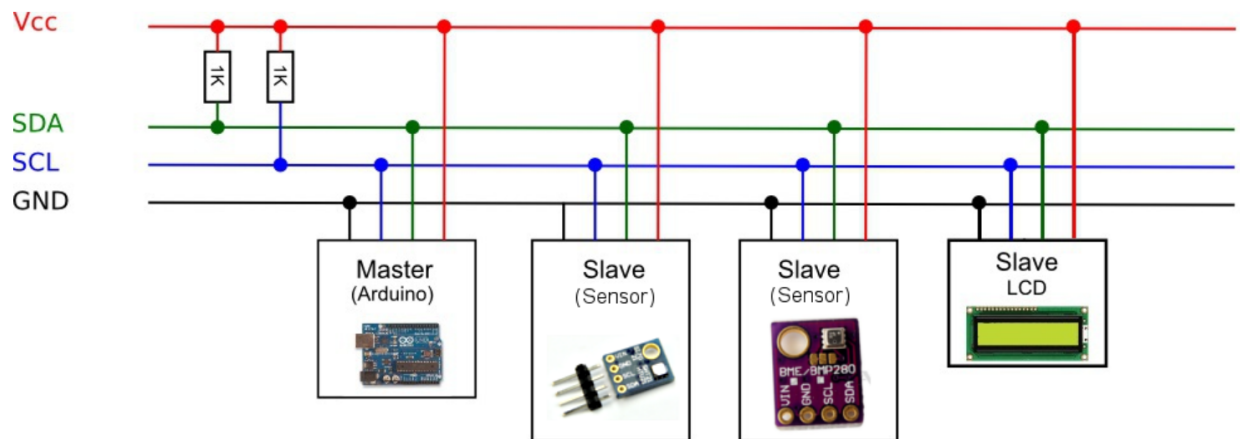
Ключевые понятия

Определение

I2C (Inter-Integrated Circuit) — синхронная двухпроводная шина, на которой устройства используют общие линии SDA и SCL и различаются адресами.

Le Bus I2C

Philips en 1982



- 2 fils de connexion (SCL : horloge, SDA: donnée)
- 127 adresses ou esclave possibles
- Vitesse 100kb/s à 400kb/s (Arduino)
- Transmission synchrone

14/21

Рис. 1 – Многоточечная топология шины I2C с линиями SDA и SCL

Интерфейс использует две линии:

- SDA (Serial Data) — двунаправленная линия данных;
- SCL (Serial Clock) — линия тактирования, которую формирует контроллер. Периферийное устройство может удерживать SCL в низком уровне при clock stretching.

Обмен начинается условием START. После семи бит адреса передаётся направление обмена, а принимающая сторона подтверждает байт сигналом ACK либо отказывается от него сигналом NACK. Обмен завершается условием STOP. В этой работе используется стандартная частота 100 кГц.

Определение

TWI (Two-Wire Interface) — используемое в документации Allwinner название аппаратного контроллера, который Linux представляет через подсистему I2C.

Линии имеют выходы с открытым стоком: устройства могут притягивать их к низкому уровню, а высокий уровень создают внешние подтягивающие резисторы. Несколько модулей могут содержать параллельные подтяжки, поэтому перед подключением нужно проверить схему конкретного дисплея.

На 40-контактный разъём Lichee RV Dock выведена смешанная группа I2C2 из банков PE и PG:

Сигнал	Вывод Allwinner D1	Физический контакт
TWI2_SCL	PE12	38
TWI2_SDA	PG15	12

Sipeed Lichee RV DOCK
pinout diagram

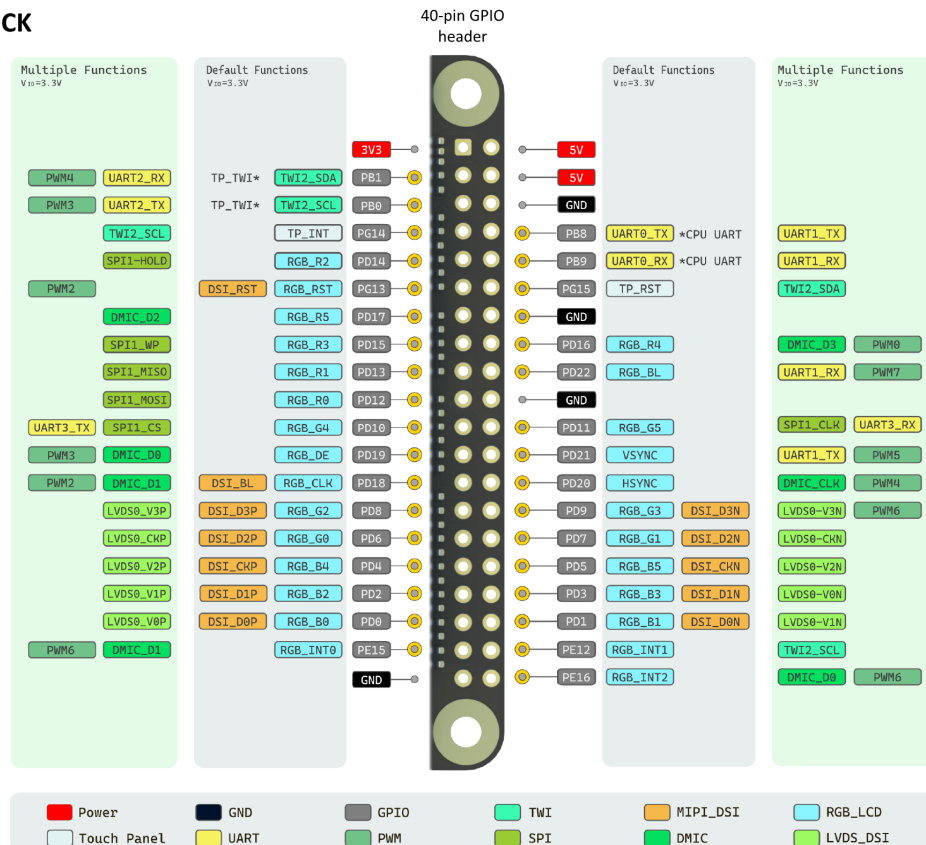


Рис. 2 – Распиновка Lichee RV Dock: линии TWI2_SDA и TWI2_SCL

Внимание

Выход PG15 также подписан как TP_RST, а PE12 — как линия RGB. Перед активацией I2C2 убедитесь, что конфликтующие узлы touchscreen и RGB не активированы в загруженном DTB.

Порядок выполнения

Активация I2C2 в Device Tree

В Linux 6.16 группа i2c2_perpg_pins уже объявлена в sun20i-d1.dtsi:

```
/omit-if-no-ref/
i2c2_perpg_pins: i2c2-perpg-pins {
    pins = "PE12", "PG15";
    function = "i2c2";
};
```

Не изменяйте общий DTSI и не перезаписывайте SPI-комплект. Создайте отдельный файл sun20i-d1-lichee-rv-dock-spi-i2c-lab.dts, который одновременно сохраняет SPI1 из лабораторной №7 и активирует I2C2:

```
// SPDX-License-Identifier: (GPL-2.0+ OR MIT)

#include "sun20i-d1-lichee-rv-dock.dts"

/ {
    aliases {
        i2c2 = &i2c2;
        spi1 = &spi1;
    };
};

&pio {
    /omit-if-no-ref/
    spi1_pd_lab_pins: spi1-pd-lab-pins {
        pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
        function = "spi1";
    };
};

&i2c2 {
    clock-frequency = <1000000>;
    pinctrl-0 = <&i2c2_pepg_pins>;
    pinctrl-names = "default";
    status = "okay";
};

&spi1 {
    pinctrl-0 = <&spi1_pd_lab_pins>;
    pinctrl-names = "default";
    status = "okay";

    spidev@0 {
        compatible = "rohm,dh2228fv";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

Alias `i2c2` закрепляет контроллер за `/dev/i2c-2`. Без alias номер адаптера может назначаться динамически, поэтому его всё равно нужно подтвердить после загрузки.

Добавьте DTB в `arch/riscv/boot/dts/allwinner/Makefile`:

```
dtb-$(CONFIG_ARCH_SUNXI) += sun20i-d1-lichee-rv-dock-spi-i2c-lab.dtb
```

Настройка и сборка ядра

Для этой конфигурации нужны три параметра I2C:

```
CONFIG_I2C=y
CONFIG_I2C_CHARDEV=y
CONFIG_I2C_MV64XXX=y
```

CONFIG_I2C_MV64XXX включает драйвер контроллера Allwinner D1, а CONFIG_I2C_CHARDEV создаёт пользовательский интерфейс /dev/i2c-*. CONFIG_I2C_MUX для прямого подключения SSD1306 не требуется, хотя может оставаться модулем в общей конфигурации.

Скопируйте проверенную конфигурацию лабораторной №7, измените локальную версию ядра и соберите отдельный комплект:

```
KERNEL_SRC="$HOME/kernel-source-6.16"
BUILD_DIR="$HOME/kernel-out-spi-i2c"

mkdir -p "$BUILD_DIR"
cp "$HOME/kernel-out-spi/.config" "$BUILD_DIR/.config"
"$KERNEL_SRC/scripts/config" --file "$BUILD_DIR/.config" \
  --set-str LOCALVERSION "-spi-i2c-lab" \
  -e I2C -e I2C_CHARDEV -e I2C_MV64XXX
make -C "$KERNEL_SRC" O="$BUILD_DIR" ARCH=riscv \
  CROSS_COMPILE=riscv64-linux-gnu- olddefconfig
make -C "$KERNEL_SRC" O="$BUILD_DIR" ARCH=riscv \
  CROSS_COMPILE=riscv64-linux-gnu- -j"$(nproc)" Image dtbs
```

Проверьте артефакты и итоговую конфигурацию:

```
grep -E 'CONFIG_(I2C|I2C_CHARDEV|I2C_MV64XXX)=' "$BUILD_DIR/.config"
ls -lh "$BUILD_DIR/arch/riscv/boot/Image"
ls -lh "$BUILD_DIR/arch/riscv/boot/dts/allwinner/"*spi-i2c-lab.dtb
```

Скопируйте Image и DTB на BOOT под отдельными именами. Новую запись сделайте основной, но сохраните записи spi-lab и known-good. Используйте фактический PARTUUID ROOTFS и не добавляйте ожидание меню: на проверенной плате U-Boot запускает 16-секундный watchdog.

```
default spi-i2c-lab

label spi-i2c-lab
  menu label Linux 6.16 PREEMPT_RT with SPI and I2C
  kernel /Image-6.16.0-spi-i2c-lab
  fdt /sun20i-d1-lichee-rv-dock-spi-i2c-lab.dtb
  append root=PARTUUID=PARTUUID_ROOTFS rootwait loglevel=7 console=ttyS0,115200

label spi-lab
  menu label Linux 6.16 PREEMPT_RT with SPI
  kernel /Image-6.16.0-spi-lab
  fdt /sun20i-d1-lichee-rv-dock-spi-lab.dtb
  append root=PARTUUID=PARTUUID_ROOTFS rootwait loglevel=7 console=ttyS0,115200
```

```
label known-good
menu label Linux 6.16 PREEMPT_RT known-good
kernel /Image
fdt /sun20i-d1-lichee-rv-dock.dtb
append root=PARTUUID=PARTUUID_ROOTFS rootwait loglevel=7 console=ttyS0,115200
```

Проверка I2C-адаптера

После загрузки подтвердите ядро, параметры конфигурации, устройства и binding драйверов:

```
uname -r
zcat /proc/config.gz | \
  grep -E 'CONFIG_(I2C|I2C_CHARDEV|I2C_MV64XXX|SPI_SPIDEV)= '
ls -l /dev/i2c-* /dev/spidev*
i2cdetect -l
readlink -f /sys/bus/platform/devices/2502800.i2c/driver
readlink -f /sys/bus/spi/devices/spi1.0/driver
dmesg | grep -iE 'i2c|spi|timeout|error'
```

Проверка

Ожидаемый результат. Загружено ядро 6.16.0-spi-i2c-lab, существуют /dev/i2c-2 и /dev/spidev1.0, I2C2 привязан к mv64xxx_i2c, а SPI-устройство — к spidev.

До подключения OLED ограничьте сканирование двумя допустимыми адресами:

```
i2cdetect -y 2 0x3c 0x3d
```

Без дисплея обе позиции должны содержать --. Не сканируйте неизвестную шину целиком: некоторые устройства реагируют на используемые i2cdetect пробные команды изменением состояния.

Подключение SSD1306

Штатно выключите систему, полностью снимите питание и подключите дисплей:

SSD1306	Lichee RV Dock	Физический контакт
VCC	3.3V	1
GND	GND	6
SCL	TWI2_SCL (PE12)	38
SDA	TWI2_SDA (PG15)	12

После включения повторите ограниченное обнаружение:

```
i2cdetect -y 2 0x3c 0x3d
```

Проверка

Ожидаемый результат. Для проверенного SSD1306 128×64 отображается адрес 3с, а 3d остаётся - -. Если адрес отличается, используйте фактическое значение только после сверки с документацией модуля.

Вывод имени на дисплей

Проверенная программа использует только стандартную библиотеку Python и /dev/i2c-N. Это исключает зависимость лабораторной от наличия совместимых версий Pillow и luma-oled в репозитории RISC-V. Встроенный шрифт поддерживает латинские буквы, цифры и пробел; имя и фамилию вводите латиницей, не более десяти символов в строке.

```
#!/usr/bin/env python3

import argparse
import fcntl
import os
import time

I2C_SLAVE = 0x0703
WIDTH = 128
HEIGHT = 64

FONT = {
    " ": (0x00, 0x00, 0x00, 0x00, 0x00),
    "0": (0x3E, 0x51, 0x49, 0x45, 0x3E),
    "1": (0x00, 0x42, 0x7F, 0x40, 0x00),
    "2": (0x42, 0x61, 0x51, 0x49, 0x46),
    "3": (0x21, 0x41, 0x45, 0x4B, 0x31),
    "4": (0x18, 0x14, 0x12, 0x7F, 0x10),
    "5": (0x27, 0x45, 0x45, 0x45, 0x39),
    "6": (0x3C, 0x4A, 0x49, 0x49, 0x30),
    "7": (0x01, 0x71, 0x09, 0x05, 0x03),
    "8": (0x36, 0x49, 0x49, 0x49, 0x36),
    "9": (0x06, 0x49, 0x49, 0x29, 0x1E),
    "A": (0x7E, 0x11, 0x11, 0x11, 0x7E),
    "B": (0x7F, 0x49, 0x49, 0x49, 0x36),
    "C": (0x3E, 0x41, 0x41, 0x41, 0x22),
    "D": (0x7F, 0x41, 0x41, 0x22, 0x1C),
    "E": (0x7F, 0x49, 0x49, 0x49, 0x41),
    "F": (0x7F, 0x09, 0x09, 0x09, 0x01),
    "G": (0x3E, 0x41, 0x49, 0x49, 0x7A),
    "H": (0x7F, 0x08, 0x08, 0x08, 0x7F),
    "I": (0x41, 0x41, 0x7F, 0x41, 0x41),
    "J": (0x20, 0x40, 0x41, 0x3F, 0x01),
    "K": (0x7F, 0x08, 0x14, 0x22, 0x41),
    "L": (0x7F, 0x40, 0x40, 0x40, 0x40),
```

```

"M": (0x7F, 0x02, 0x0C, 0x02, 0x7F),
"N": (0x7F, 0x04, 0x08, 0x10, 0x7F),
"O": (0x3E, 0x41, 0x41, 0x41, 0x3E),
"P": (0x7F, 0x09, 0x09, 0x09, 0x06),
"Q": (0x3E, 0x41, 0x51, 0x21, 0x5E),
"R": (0x7F, 0x09, 0x19, 0x29, 0x46),
"S": (0x26, 0x49, 0x49, 0x49, 0x32),
"T": (0x01, 0x01, 0x7F, 0x01, 0x01),
"U": (0x3F, 0x40, 0x40, 0x40, 0x3F),
"V": (0x1F, 0x20, 0x40, 0x20, 0x1F),
"W": (0x3F, 0x40, 0x38, 0x40, 0x3F),
"X": (0x63, 0x14, 0x08, 0x14, 0x63),
"Y": (0x07, 0x08, 0x70, 0x08, 0x07),
"Z": (0x61, 0x51, 0x49, 0x45, 0x43),
}

def command(fd, *values):
    os.write(fd, bytes((0x00, *values)))

def initialize(fd):
    command(
        fd,
        0xAE, 0xD5, 0x80, 0xA8, 0x3F, 0xD3, 0x00, 0x40,
        0x8D, 0x14, 0x20, 0x00, 0xA1, 0xC8, 0xDA, 0x12,
        0x81, 0xCF, 0xD9, 0xF1, 0xDB, 0x40, 0xA4, 0xA6, 0xAF,
    )

def draw_text(buffer, text, page):
    text = text.upper()
    unsupported = sorted(set(text) - FONT.keys())
    if unsupported:
        raise ValueError(f"unsupported characters: {''.join(unsupported)}")
    if len(text) > 10:
        raise ValueError("each line must contain at most 10 characters")

    scale = 2
    x = (WIDTH - len(text) * 6 * scale) // 2
    for character in text:
        for column in (*FONT[character], 0x00):
            expanded = 0
            for bit in range(7):
                if column & (1 << bit):
                    expanded |= 0x03 << (bit * scale)
            for repeat in range(scale):
                buffer[(page * WIDTH) + x + repeat] = expanded & 0xFF
                buffer[((page + 1) * WIDTH) + x + repeat] = expanded >> 8

```

```

        x += scale

def write_frame(fd, buffer):
    command(fd, 0x21, 0, WIDTH - 1, 0x22, 0, HEIGHT // 8 - 1)
    for offset in range(0, len(buffer), 16):
        os.write(fd, bytes((0x40,)) + buffer[offset:offset + 16])

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("first_name")
    parser.add_argument("last_name")
    parser.add_argument("--port", type=int, default=2)
    parser.add_argument("--address", type=lambda value: int(value, 0),
        ↪ default=0x3C)
    parser.add_argument("--updates", type=int, default=20)
    parser.add_argument("--seconds", type=float, default=10)
    return parser.parse_args()

def main():
    args = parse_args()
    path = f"/dev/i2c-{args.port}"
    buffer = bytearray(WIDTH * HEIGHT // 8)
    draw_text(buffer, args.first_name, 1)
    draw_text(buffer, args.last_name, 5)

    fd = os.open(path, os.O_RDWR)
    try:
        fcntl.ioctl(fd, I2C_SLAVE, args.address)
        initialize(fd)
        for _ in range(args.updates):
            write_frame(fd, buffer)
        print(
            f"updates={args.updates} errors=0 address={args.address:#04x} "
            f"adapter={path}"
        )
        if args.seconds > 0:
            time.sleep(args.seconds)
            write_frame(fd, bytearray(len(buffer)))
    finally:
        os.close(fd)

if __name__ == "__main__":
    main()

```

Запустите программу, передав транслитерацию имени и фамилии:

```
python3 display_name.py IVAN PETROV
```

Программа записывает 20 полных кадров, показывает две строки 10 секунд и очищает дисплей. Для фотографии результата можно временно отключить автоматическую очистку:

```
python3 display_name.py IVAN PETROV --seconds 0
```

Проверка

Ожидаемый результат. Программа сообщает updates=20 errors=0 address=0x3c adapter=/dev/i2c-2, обе строки полностью отображаются без артефактов.

Отрицательные проверки

Проверьте неверный адрес и отсутствующий адаптер:

```
python3 display_name.py IVAN PETROV --address 0x3d --seconds 0  
echo "wrong_address_rc=$?"
```

```
python3 display_name.py IVAN PETROV --port 99 --seconds 0  
echo "missing_adapter_rc=$?"
```

Обе команды должны завершиться ненулевым кодом. Неверный адрес приводит к ошибке записи/NACK, а отсутствующий адаптер — к No such file or directory. После отрицательных тестов повторите штатный запуск и убедитесь, что дисплей снова работает.

Повторите короткий SPI loopback из лабораторной №7. Одновременное наличие /dev/i2c-2 и /dev/spidev1.0 недостаточно: необходимо доказать обмен по обоим интерфейсам.

Результаты проверочного стенда

На Lichee RV Dock с Linux 6.16.0-spi-i2c-lab PREEMPT_RT и ALT Regular 20260316 получены следующие результаты:

Проверка	Фактический результат
Image	36026880 байт, SHA-256 6bf0a0b7845886f0840febd02e586f7389b859d2
DTB	20963 байта, SHA-256 b3f01e58a8678fe3d27664576c8ea58326e1c62b
I2C adapter	/dev/i2c-2, mv64xxx_i2c adapter
SSD1306	ACK на 0x3C, NACK на 0x3D
Вывод текста	IVAN/PETROV, 20 полных кадров, 0 ошибок, без артефактов

Проверка	Фактический результат
Отсутствующий адаптер	/dev/i2c-99: ненулевой код, No such file or directory
SPI после добавления I2C	1024 байта на 1 МГц, csp = 0

Задание

1. Подтвердите по исходному дереву и распиновке соответствие PE12/PG15 контроллеру I2C2 и физическим контактам 38/12.
2. Создайте отдельный совместный DTS, сохранив SPI1 из лабораторной №7.
3. Закрепите I2C2 за номером 2 через alias и задайте частоту 100 кГц.
4. Включите CONFIG_I2C, CONFIG_I2C_CHARDEV и CONFIG_I2C_MV64XXX встроено.
5. Соберите отдельные Image и DTB и добавьте загрузочную запись по PARTUUID, сохранив spi-lab и known-good.
6. Подтвердите /dev/i2c-2, /dev/spidev1.0 и binding обоих контроллеров.
7. Без OLED подтвердите отсутствие ответов на 0x3C/0x3D, затем подключите модуль при снятом питании и обнаружьте 0x3C.
8. Выведите имя и фамилию латиницей, выполните 20 полных обновлений без ошибок.
9. Проведите отрицательные тесты с 0x3D и /dev/i2c-99, затем повторите штатный запуск.
10. Подтвердите сохранение SPI loopback и продемонстрируйте дисплей преподавателю.

Контрольные вопросы

1. Почему контроллер I2C на Lichee RV Dock обозначен как TWI?
2. Как организованы семибитная адресация, бит направления и ACK/NACK?
3. Почему SDA и SCL требуют подтягивающих резисторов?
4. Может ли периферийное устройство влиять на SCL?
5. Какие параметры ядра необходимы для I2C на Allwinner D1 и для чего нужен CONFIG_I2C_CHARDEV?
6. Зачем в Device Tree используется alias i2c2 и почему фактический номер всё равно проверяют?
7. Как безопасно проверить адрес SSD1306, не сканируя неизвестную шину целиком?
8. Чем отличаются отсутствие адаптера, NACK по неверному адресу и ошибка прав доступа?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- подтверждённое соответствие PE12/PG15 контактам 38/12;
- совместный SPI+I2C DTS, параметры ядра и загрузочную запись;
- вывод `uname -r`, `i2cdetect -l`, `binding` драйверов и наличие `/dev/i2c-2` и `/dev/spidev1.0`;
- схему или фотографию безопасного подключения SSD1306;
- ограниченный скан адресов до и после подключения;
- исходный код, строку результата 20 обновлений и фотографию текста;
- результаты отрицательных тестов и повторного штатного запуска;
- подтверждение сохранения SPI loopback;
- ответы на контрольные вопросы и краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.